

Decision Trees - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Option A: Binary Classification (Loan Approval Dataset)

Step 1: Importing Libraries

Import necessary libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Initial Data Loading & Exploration

Load the dataset

```
data = pd.read_csv("loan_approval_dataset.csv")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Check the shape of the dataset

```
print("Dataset Shape:", data.shape)
```

Output:

```
Dataset Shape: (4269, 13)
```

Fix column spacing

```
data.columns = data.columns.str.strip()

# Print columns in vertical form
print("\n\nColumns (Vertical List):")
for col in data.columns:
    print(col)
```

Output:

Columns (Vertical List):

```

loan_id
no_of_dependents
education
self_employed
income_annum
loan_amount
loan_term
cibil_score
residential_assets_value
commercial_assets_value
luxury_assets_value
bank_asset_value
loan_status

```

Display first 5 rows

```

print("First 5 rows of the dataset:")
data.head()

```

Output:

First 5 rows of the dataset:

loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets	commercial_assets_	luxury_assets_valu	bank_asset_value	loan_status
0 1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	Approved
1 2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	Rejected
2 3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	Rejected
3 4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	Rejected
4 5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	Rejected

Display last 5 rows

```

print("Last 5 rows of the dataset:")
data.tail()

```

Output:

Last 5 rows of the dataset:

loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term
cibil_score	residential_assets	commercial_assets_	luxury_assets_valu	bank_asset_value		
loan_status						
4264	4265	5	Graduate	Yes	1000000	2300000
317		2800000	500000	3300000	800000	12
Rejected						
4265	4266	0	Not Graduate	Yes	3300000	11300000
559		4200000	2900000	11000000	1900000	20
Approved						
4266	4267	2	Not Graduate	No	6500000	23900000
457		1200000	12400000	18100000	7300000	18
Rejected						
4267	4268	1	Not Graduate	No	4100000	12800000
780		8200000	700000	14100000	5800000	8
Approved						
4268	4269	1	Graduate	No	9200000	29700000
607		17800000	11800000	35700000	12000000	10
Approved						

Step 3: Data Preprocessing & Information

Target Column Encoding

```
data.columns = data.columns.str.strip()
data['loan_status'] = data['loan_status'].str.strip().map({
    'Approved': 1,
    'Rejected': 0
})

print("Target column loan_status encoded successfully")
data.head()
```

Output:

Target column loan_status encoded successfully

loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term
cibil_score	residential_assets	commercial_assets_	luxury_assets_valu	bank_asset_value		
loan_status						
0	1	2	Graduate	No	9600000	29900000
778		2400000	17600000	22700000	8000000	1
1	2	0	Not Graduate	Yes	4100000	12200000
417		2700000	2200000	8800000	3300000	0
2	3	3	Graduate	No	9100000	29700000
506		7100000	4500000	33300000	12800000	0
3	4	3	Graduate	No	8200000	30700000
467		18200000	3300000	23300000	7900000	0
4	5	5	Not Graduate	Yes	9800000	24200000
382		12400000	8200000	29400000	5000000	0

Data Information

```
print("All Required Information Related To Data: ")
data.info()
```

Output:

All Required Information Related To Data:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4269 entries, 0 to 4268
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	loan_id	4269 non-null	int64
1	no_of_dependents	4269 non-null	int64
2	education	4269 non-null	object
3	self_employed	4269 non-null	object
4	income_annum	4269 non-null	int64
5	loan_amount	4269 non-null	int64
6	loan_term	4269 non-null	int64
7	cibil_score	4269 non-null	int64
8	residential_assets_value	4269 non-null	int64
9	commercial_assets_value	4269 non-null	int64
10	luxury_assets_value	4269 non-null	int64
11	bank_asset_value	4269 non-null	int64
12	loan_status	4269 non-null	int64

```
dtypes: int64(11), object(2)
```

```
memory usage: 433.7+ KB
```

Check for missing (null) values in each column

```
print("Missing values in each column:")
print(data.isnull().sum())
```

Output:

Missing values in each column:

loan_id	0
no_of_dependents	0
education	0
self_employed	0
income_annum	0
loan_amount	0
loan_term	0
cibil_score	0
residential_assets_value	0
commercial_assets_value	0
luxury_assets_value	0
bank_asset_value	0
loan_status	0

```
dtype: int64
```

Step 4: Feature Engineering & Data Splitting

Define target column

```

target_col = "loan_status"
print("Target column selected:", target_col)

# Separate features and target
X = data.drop(columns=[target_col])
y = data[target_col]
print("Features (X) and target (y) separated successfully")

# Identify categorical columns
cat_cols = X.select_dtypes(include=['object']).columns.tolist()
print("Categorical columns identified:", cat_cols)

# Apply One-Hot Encoding to categorical columns
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ],
    remainder='passthrough'
)

# Transform feature data
X_transformed = preprocessor.fit_transform(X)
print("Categorical features one-hot encoded and dataset transformed successfully")

```

Output:

```

Target column selected: loan_status
Features (X) and target (y) separated successfully
Categorical columns identified: ['education', 'self_employed']
Categorical features one-hot encoded and dataset transformed successfully

```

Split the dataset into training and testing sets

```

X_train, X_test, y_train, y_test = train_test_split(
    X_transformed,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Dataset successfully split into training and testing sets")
print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)

```

Output:

```

Dataset successfully split into training and testing sets
Training set size: (3415, 14)
Testing set size: (854, 14)

```

Step 5: Model Training & Evaluation (Gini Criterion)

Initialize and train Decision Tree model using Gini impurity

```
dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)
dt_model.fit(X_train, y_train)
print("Decision Tree model trained successfully using Gini impurity")
```

Output:

Decision Tree model trained successfully using Gini impurity

Evaluate Decision Tree model (Gini impurity)

```
y_pred = dt_model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix (Gini, Full Tree):")
print(confusion_matrix(y_test, y_pred))
```

Output:

Accuracy (Gini, Full Tree): 0.9789227166276346
Confusion Matrix (Gini, Full Tree):
[[313 10]
[8 523]]

Compute confusion matrix for Decision Tree model (Gini impurity)

```
print("Calculating confusion matrix (Gini)...")
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix calculated successfully")
```

Output:

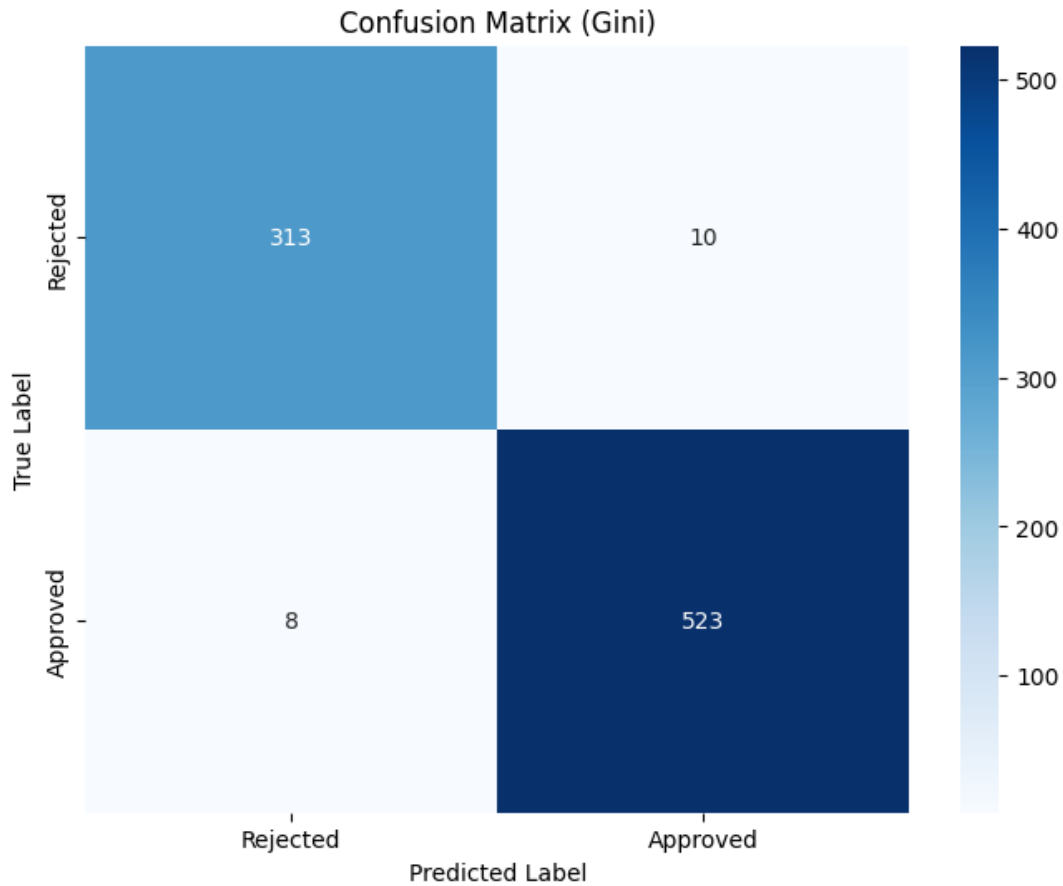
Calculating confusion matrix (Gini)...
Confusion matrix calculated successfully

Plot confusion matrix for Decision Tree model (Gini impurity)

```
print("Displaying confusion matrix plot (Gini)...")
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Rejected', 'Approved'],
            yticklabels=['Rejected', 'Approved'])
plt.title('Confusion Matrix (Gini)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix plot displayed successfully")
```

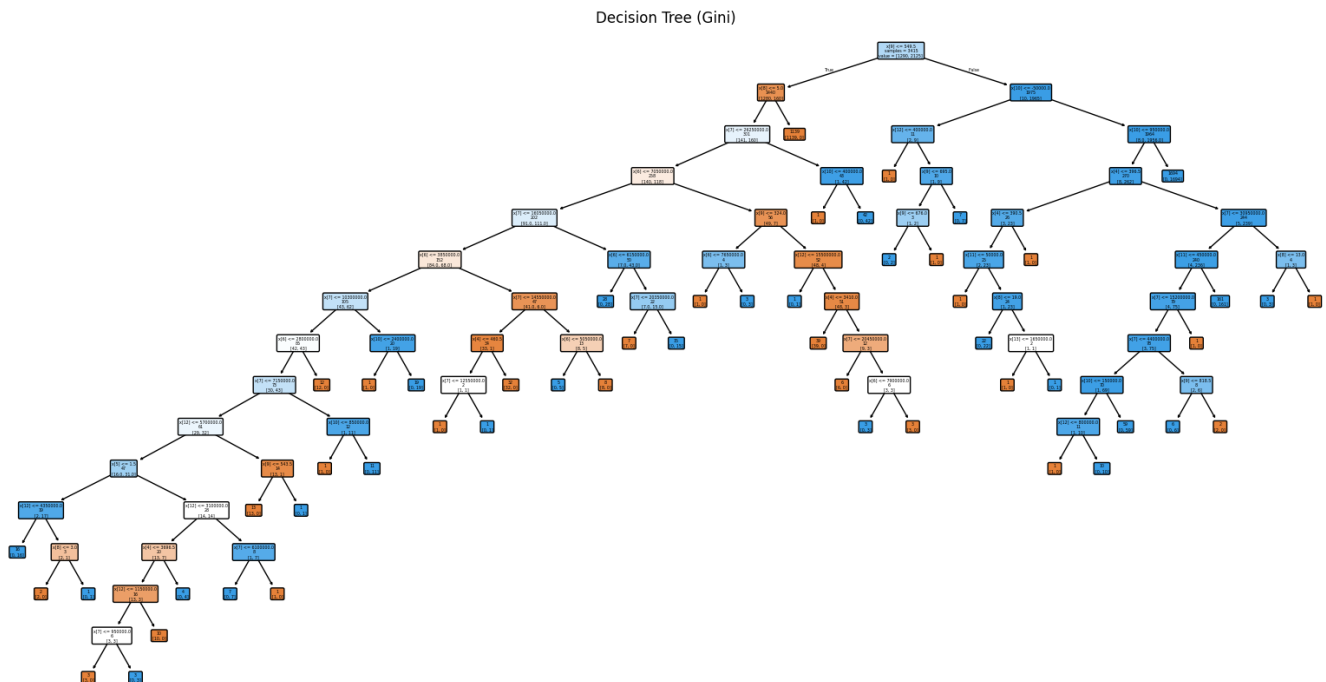
Output:

Displaying confusion matrix plot (Gini)...Confusion matrix plot displayed successfully



Visualize the Gini Decision Tree

```
plt.figure(figsize=(20, 10))
plot_tree(dt_model, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini)")
plt.show()
```



Step 6: Model Training & Evaluation (Entropy Criterion)

Train and evaluate Decision Tree model using Entropy

```
dt_model_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_model_entropy.fit(X_train, y_train)
print("Decision Tree model trained successfully using Entropy")

# Predict on test data
y_pred_entropy = dt_model_entropy.predict(X_test)
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
print("\nConfusion Matrix (Entropy, Full Tree):")
print(confusion_matrix(y_test, y_pred_entropy))
```

Output:

```
Decision Tree model trained successfully using EntropyAccuracy (Entropy, Full Tree):
0.9789227166276346
Confusion Matrix (Entropy, Full Tree):
[[312  11]
 [  7 524]]
```

Compute confusion matrix for Decision Tree model (Entropy)

```
print("Calculating confusion matrix (Entropy)...")
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
print("Confusion matrix (Entropy) calculated successfully")
```

Output:

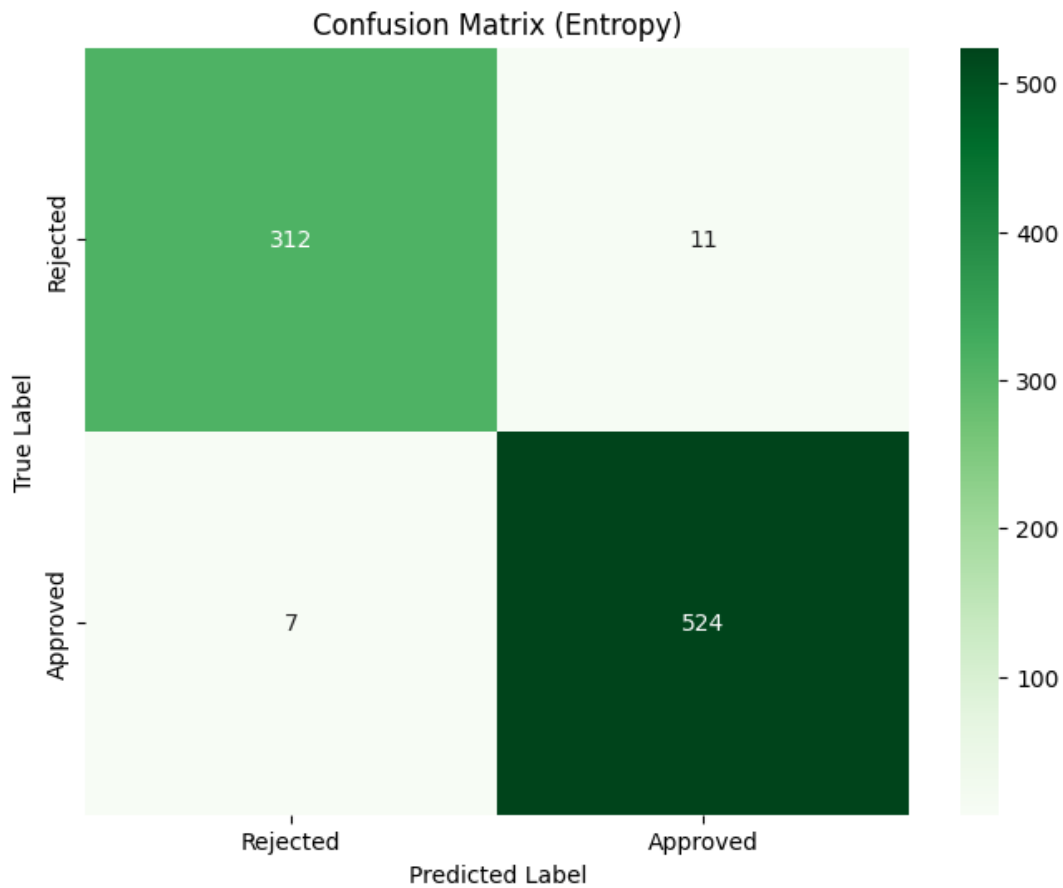
```
Calculating confusion matrix (Entropy)...
Confusion matrix (Entropy) calculated successfully
```

Plot confusion matrix for Decision Tree model (Entropy)

```
target_names = ['Rejected', 'Approved']
plt.figure(figsize=(8, 6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
            yticklabels=target_names)
plt.title('Confusion Matrix (Entropy)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
print("Confusion matrix (Entropy) plot displayed successfully")
```

Output:

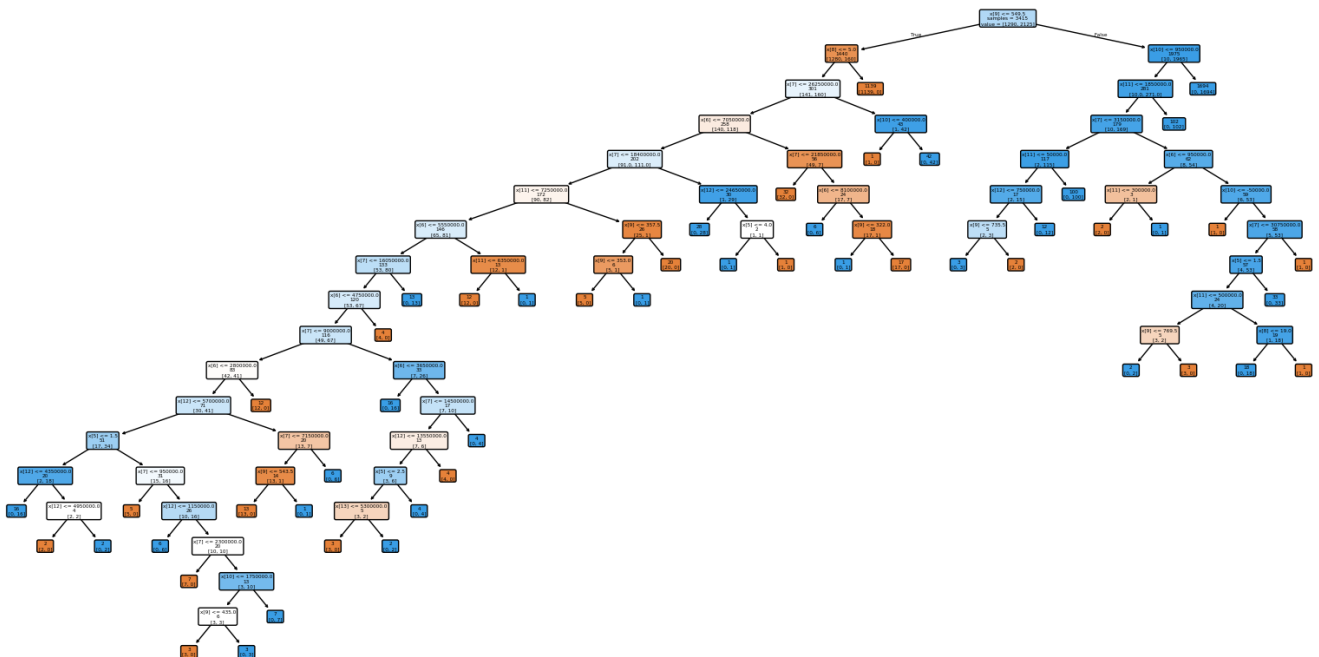
```
Displaying confusion matrix plot (Entropy)...Confusion matrix (Entropy) plot displayed
successfully
```

Visualize the Entropy Decision Tree

```
plt.figure(figsize=(20, 10))
plot_tree(dt_model_entropy, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy)")
plt.show()
```

Decision Tree (Entropy)



Step 7: Decision Tree Depth Analysis (Gini)

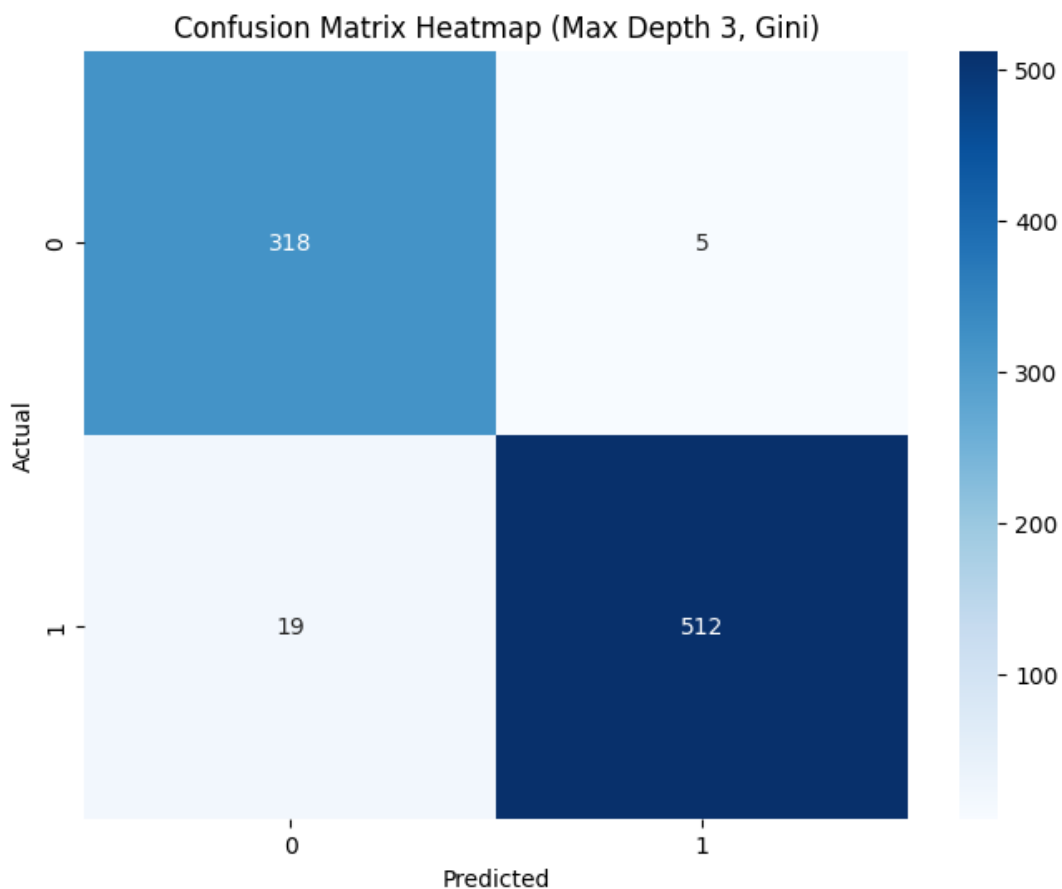
Train Decision Tree with max_depth=3 (Gini)

```
dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
dt_gini_3.fit(X_train, y_train)
y_pred_3 = dt_gini_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Gini): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

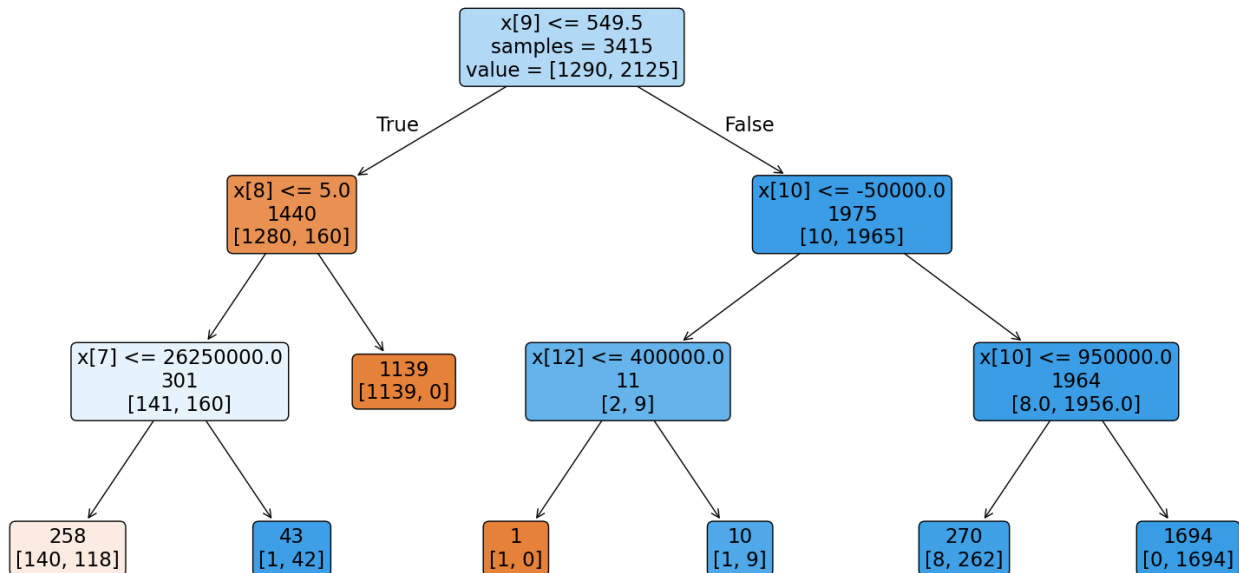
```
Accuracy (Max Depth 3, Gini): 0.9403
Confusion Matrix (Max Depth 3):
[[280  43]
 [  8 523]]
```



Plot the tree (Gini - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_3, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 3")
plt.show()
```

Decision Tree (Gini) - Max Depth 3



Train Decision Tree with max_depth=5 (Gini)

```
dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
dt_gini_5.fit(X_train, y_train)
y_pred_5 = dt_gini_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Gini): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
Accuracy (Max Depth 5, Gini): 0.9707
Confusion Matrix (Max Depth 5):
[[308  15]
 [ 10 521]]
```



Decision Tree (Gini) - Max Depth 5



Train Decision Tree with max_depth=7 (Gini)

```

dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)
dt_gini_7.fit(X_train, y_train)
y_pred_7 = dt_gini_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Gini): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Gini)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

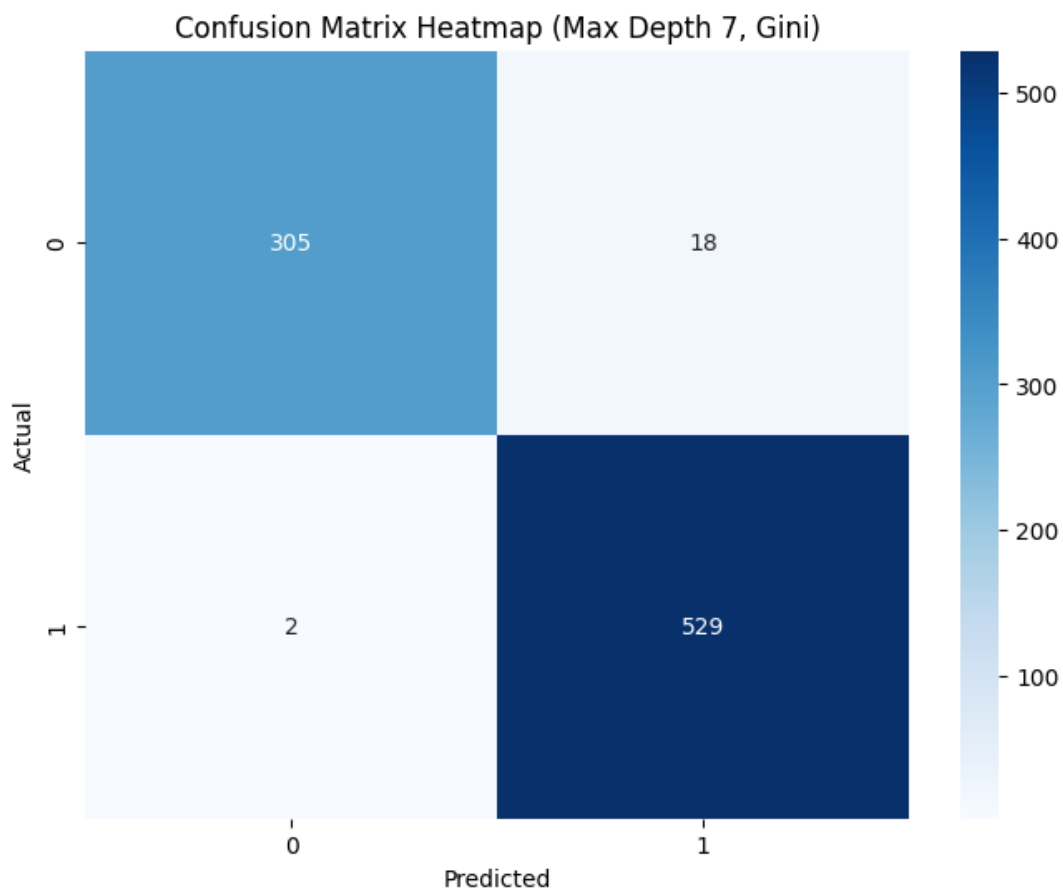
```

Output:

```

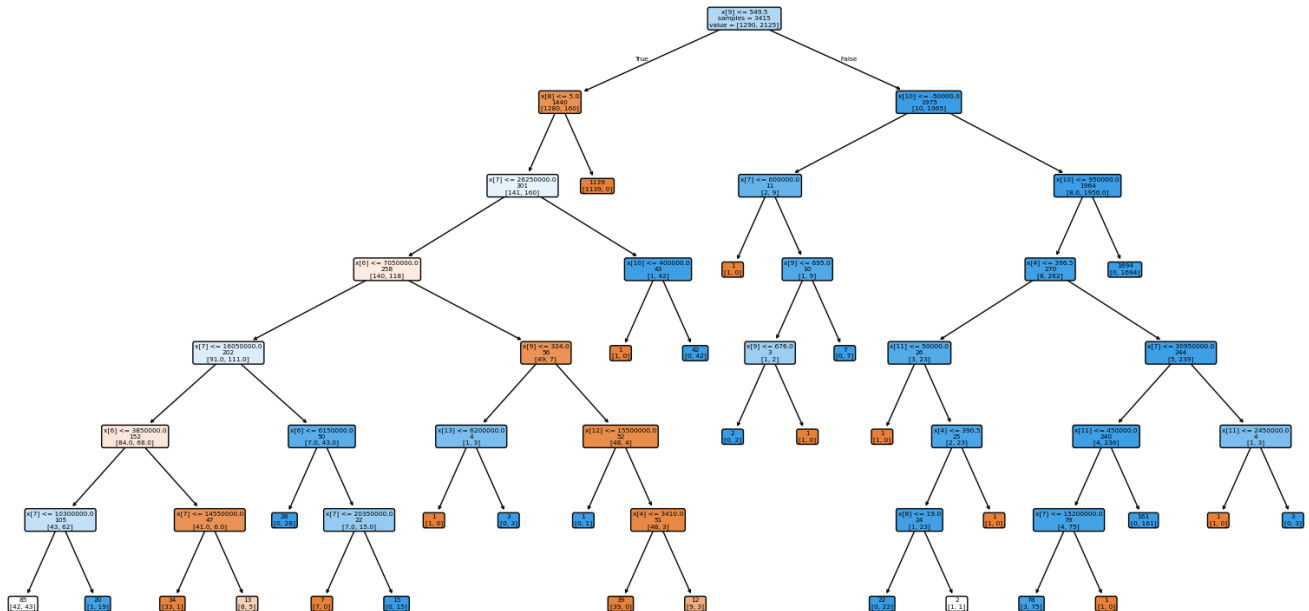
Accuracy (Max Depth 7, Gini): 0.9766
Confusion Matrix (Max Depth 7):
[[312  11]
 [  9 522]]

```

**# Plot the tree (Gini - Max Depth 7)**

```
plt.figure(figsize=(20, 10))
plot_tree(dt_gini_7, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Gini) - Max Depth 7")
plt.show()
```

Decision Tree (Gini) - Max Depth 7



Step 8: Decision Tree Depth Analysis (Entropy)

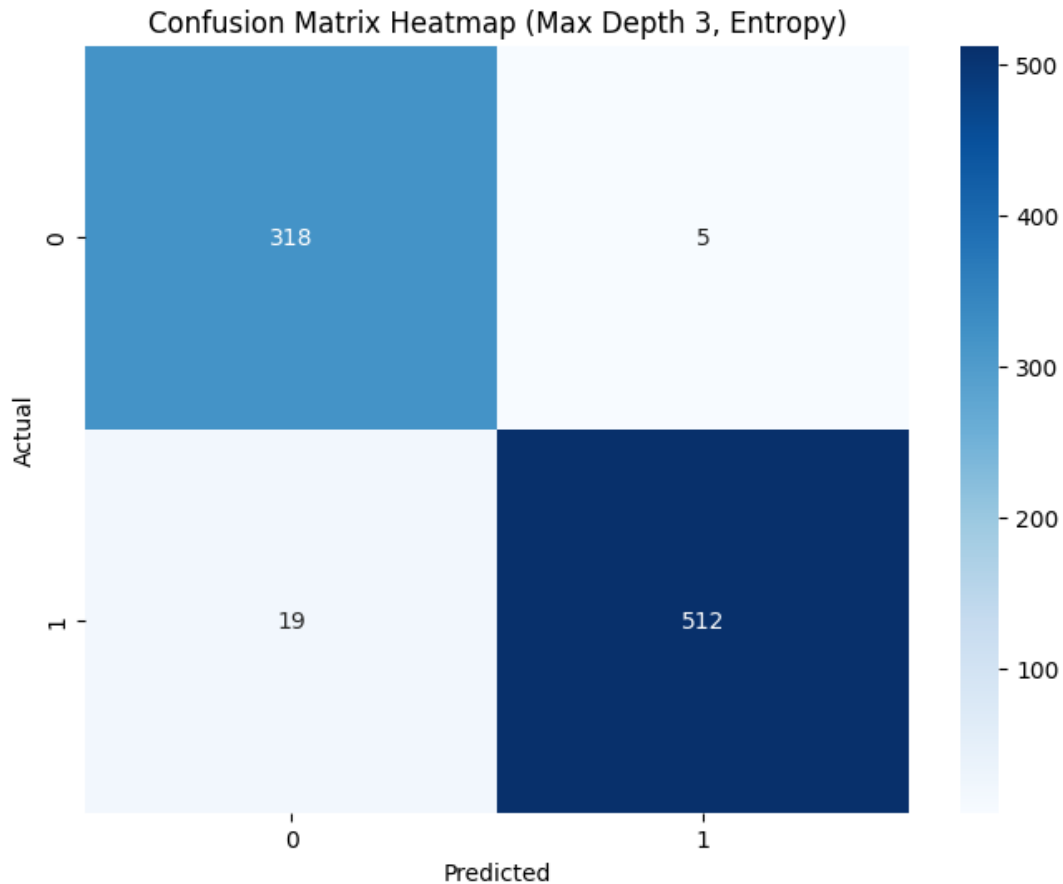
Train Decision Tree with max_depth=3 (Entropy)

```
dt_entropy_3 = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
dt_entropy_3.fit(X_train, y_train)
y_pred_3 = dt_entropy_3.predict(X_test)
accuracy_3 = accuracy_score(y_test, y_pred_3)
print(f"Accuracy (Max Depth 3, Entropy): {accuracy_3:.4f}")
print("Confusion Matrix (Max Depth 3):")
print(confusion_matrix(y_test, y_pred_3))
```

```
# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_3), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 3, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

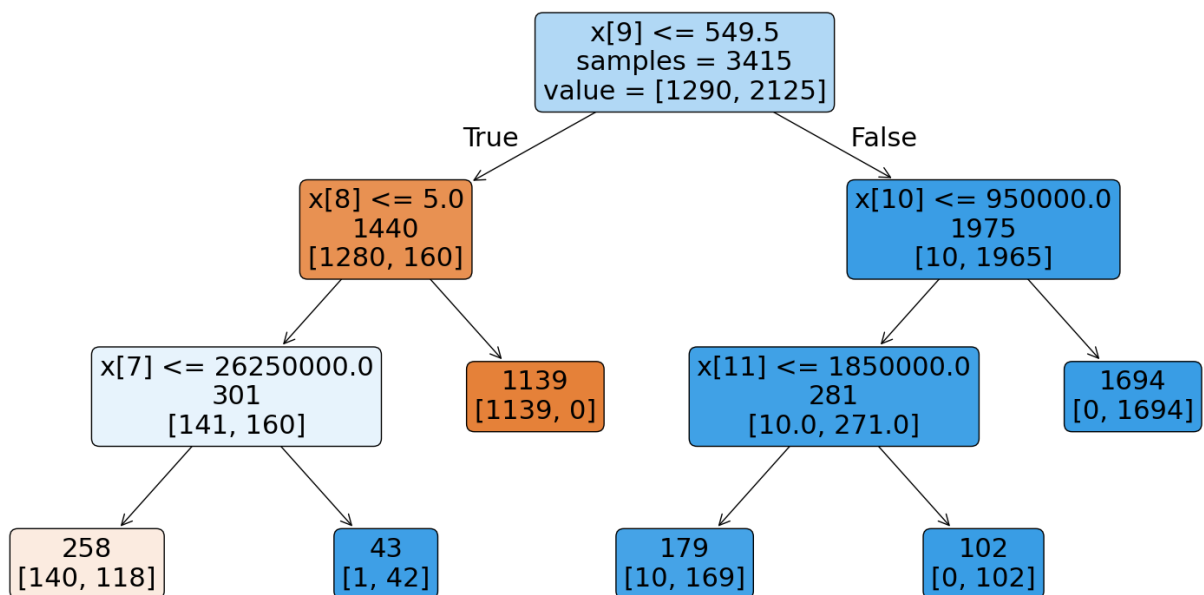
```
Accuracy (Max Depth 3, Entropy): 0.9333
Confusion Matrix (Max Depth 3):
[[275  48]
 [  9 522]]
```



Plot the tree (Entropy - Max Depth 3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_3, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 3")
plt.show()
```

Decision Tree (Entropy) - Max Depth 3



Train Decision Tree with max_depth=5 (Entropy)

```

dt_entropy_5 = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=42)
dt_entropy_5.fit(X_train, y_train)
y_pred_5 = dt_entropy_5.predict(X_test)
accuracy_5 = accuracy_score(y_test, y_pred_5)
print(f"Accuracy (Max Depth 5, Entropy): {accuracy_5:.4f}")
print("Confusion Matrix (Max Depth 5):")
print(confusion_matrix(y_test, y_pred_5))

# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_5), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 5, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

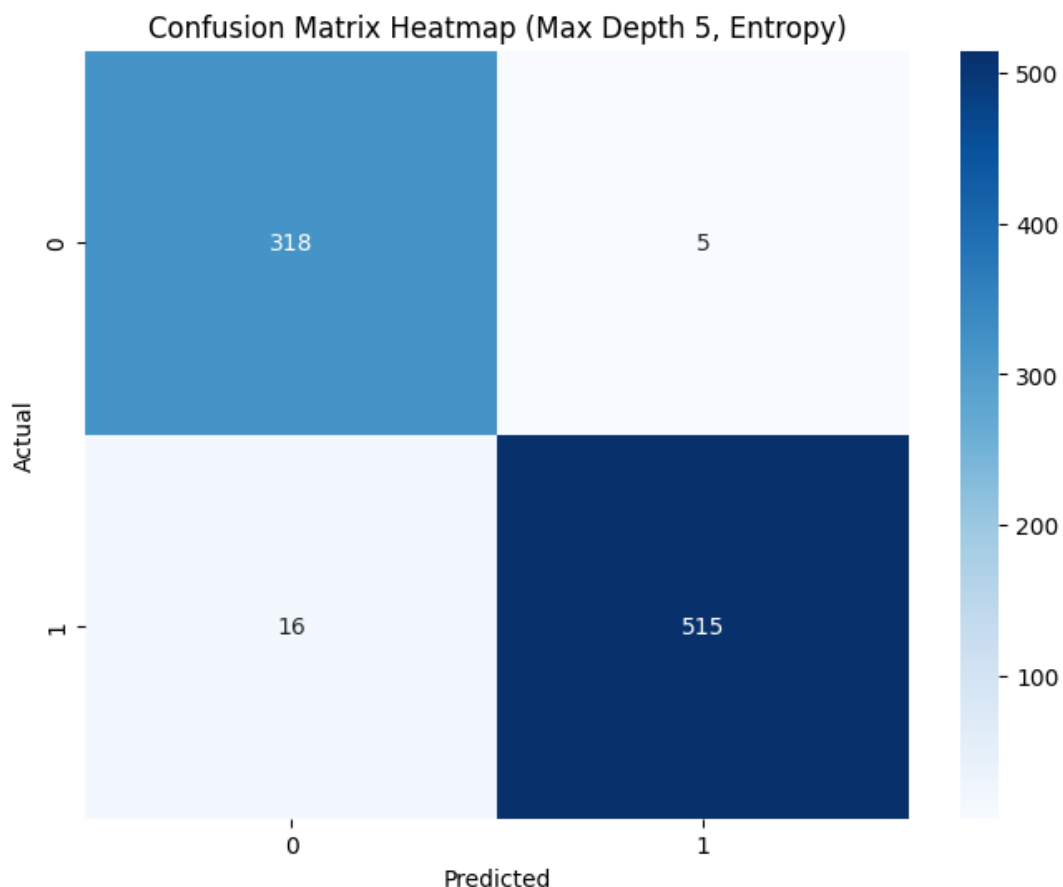
```

Output:

```

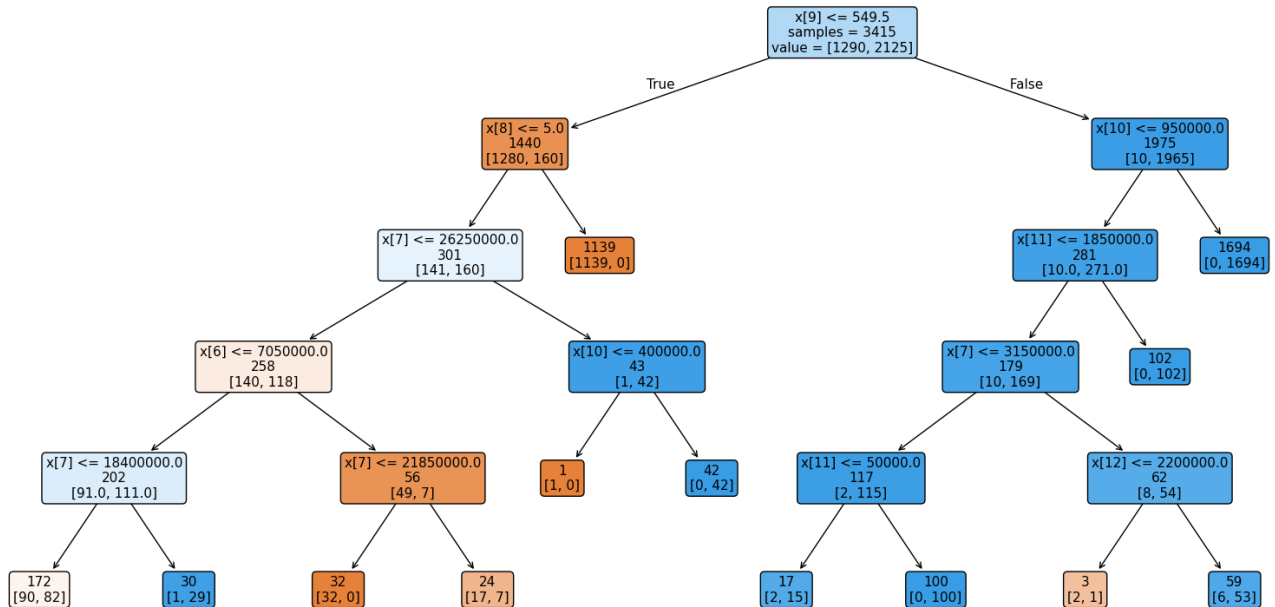
Accuracy (Max Depth 5, Entropy): 0.9637
Confusion Matrix (Max Depth 5):
[[304  19]
 [ 12 519]]

```

**# Plot the tree (Entropy - Max Depth 5)**


```
plt.figure(figsize=(20, 10))
plot_tree(dt_entropy_5, filled=True, rounded=True, label='root', impurity=False,
          proportion=False)
plt.title("Decision Tree (Entropy) - Max Depth 5")
plt.show()
```

Decision Tree (Entropy) - Max Depth 5



Train Decision Tree with max_depth=7 (Entropy)

```
dt_entropy_7 = DecisionTreeClassifier(criterion='entropy', max_depth=7, random_state=42)
dt_entropy_7.fit(X_train, y_train)
y_pred_7 = dt_entropy_7.predict(X_test)
accuracy_7 = accuracy_score(y_test, y_pred_7)
print(f"Accuracy (Max Depth 7, Entropy): {accuracy_7:.4f}")
print("Confusion Matrix (Max Depth 7):")
print(confusion_matrix(y_test, y_pred_7))
```

```
# Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_7), annot=True, fmt='d', cmap='Blues',
            xticklabels=True, yticklabels=True)
plt.title("Confusion Matrix Heatmap (Max Depth 7, Entropy)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:

```
Accuracy (Max Depth 7, Entropy): 0.9766
Confusion Matrix (Max Depth 7):
[[312  11]
 [  9 522]]
```



Decision Tree (Entropy) - Max Depth 7



Option B: Multiclass Classification (Obesity Dataset)

Step 1: Importing Libraries

Import necessary libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
print("Libraries imported successfully!")
```

Output:

```
Libraries imported successfully!
```

Step 2: Loading Data

Load the dataset

```
data = pd.read_csv('ObesityDataSet_raw_and_data_synthetic.csv')
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Check the shape of the dataset

```
print("Data Shape:")
print(data.shape)
```

Output:

```
Data Shape:
(2111, 17)
```

Display first 5 rows of the dataset

```
print("First 5 rows of the dataset:")
data.head()
```

Output:

First 5 rows of the dataset:

	Gender	Age	Height	Weight	family_history_wit	FAVC	FCVC	NCP	CAEC	SMOKE	CH20	SCC
0	Female	21.0	1.62	64.0	yes	no	2.0	3.0	Sometimes	no	2.0	
	no	0.0	1.0	no	Public_Transportat	Normal_Weight						
1	Female	21.0	1.52	56.0	yes	no	3.0	3.0	Sometimes	yes	3.0	
	yes	3.0	0.0	Sometimes	Public_Transportat	Normal_Weight						
2	Male	23.0	1.80	77.0	yes	no	2.0	3.0	Sometimes	no	2.0	
	no	2.0	1.0	Frequently	Public_Transportat	Normal_Weight						
3	Male	27.0	1.80	87.0	no	no	3.0	3.0	Sometimes	no	2.0	
	no	2.0	0.0	Frequently	Walking	Overweight_Level_I						
4	Male	22.0	1.78	89.8	no	no	2.0	1.0	Sometimes	no	2.0	
	no	0.0	0.0	Sometimes	Public_Transportat	Overweight_Level_I						

Display last 5 rows of the dataset

```
print("Last 5 rows of the dataset:")
data.tail()
```

Output:

Last 5 rows of the dataset:

	Gender	Age	Height	Weight	family_history_wit	FAVC	FCVC	NCP	CAEC	SMOKE
2106	Female	20.976842	1.710730	131.408528	yes		yes	3.0	3.0	Sometimes
	no	1.728139	no	1.676269	0.906247	Sometimes	Public_Transportat	Obesity_Type_III		
2107	Female	21.982942	1.748584	133.742943	yes		yes	3.0	3.0	Sometimes
	no	2.005130	no	1.341390	0.599270	Sometimes	Public_Transportat	Obesity_Type_III		
2108	Female	22.524036	1.752206	133.689352	yes		yes	3.0	3.0	Sometimes
	no	2.054193	no	1.414209	0.646288	Sometimes	Public_Transportat	Obesity_Type_III		
2109	Female	24.361936	1.739450	133.346641	yes		yes	3.0	3.0	Sometimes
	no	2.852339	no	1.139107	0.586035	Sometimes	Public_Transportat	Obesity_Type_III		
2110	Female	23.664709	1.738836	133.472641	yes		yes	3.0	3.0	Sometimes
	no	2.863513	no	1.026452	0.714137	Sometimes	Public_Transportat	Obesity_Type_III		

Step 3: Preprocessing

```
print("Statistical summary of the dataset:")
data.describe()
```

Output:

Statistical summary of the dataset:

	Age	Height	Weight	FCVC	NCP	CH2O	FAF	TUE
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	
mean	24.312600	1.701677	86.586058	2.419043	2.685628	2.008011	1.010298	
std	6.345968	0.093305	26.191172	0.533927	0.778039	0.612953	0.850592	
min	14.000000	1.450000	39.000000	1.000000	1.000000	1.000000	0.000000	
25%	19.947192	1.630000	65.473343	2.000000	2.658738	1.584812	0.124505	
50%	22.777890	1.700499	83.000000	2.385502	3.000000	2.000000	1.000000	
75%	26.000000	1.768464	107.430682	3.000000	3.000000	2.477420	1.666678	
max	61.000000	1.980000	173.000000	3.000000	4.000000	3.000000	3.000000	

Initialize LabelEncoder

```
le = LabelEncoder()
target_names = []
print("LabelEncoder initialized")
```

Output:

LabelEncoder initialized

Apply encoding to categorical columns

```
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])
    if col == 'NObeyesdad':
        target_names = list(le.classes_)
print("Categorical variables encoded")
```

Output:

Categorical variables encoded

Split data into training and testing sets

```
X = data.drop('NObeyesdad', axis=1)
y = data['NObeyesdad']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data split complete")
```

Output:

Data split complete

Step 4: Training (Gini)

Initialize and train Decision Tree with Gini impurity

```
model = DecisionTreeClassifier(criterion='gini')
model.fit(X_train, y_train)
print("Model trained using Gini impurity")
```

Output:

```
Model trained using Gini impurity
```

Evaluate Gini model

```
y_pred = model.predict(X_test)
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy (Gini, Full Tree): 0.9433
```

Compute confusion matrix for Gini

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix shape:", cm.shape)
```

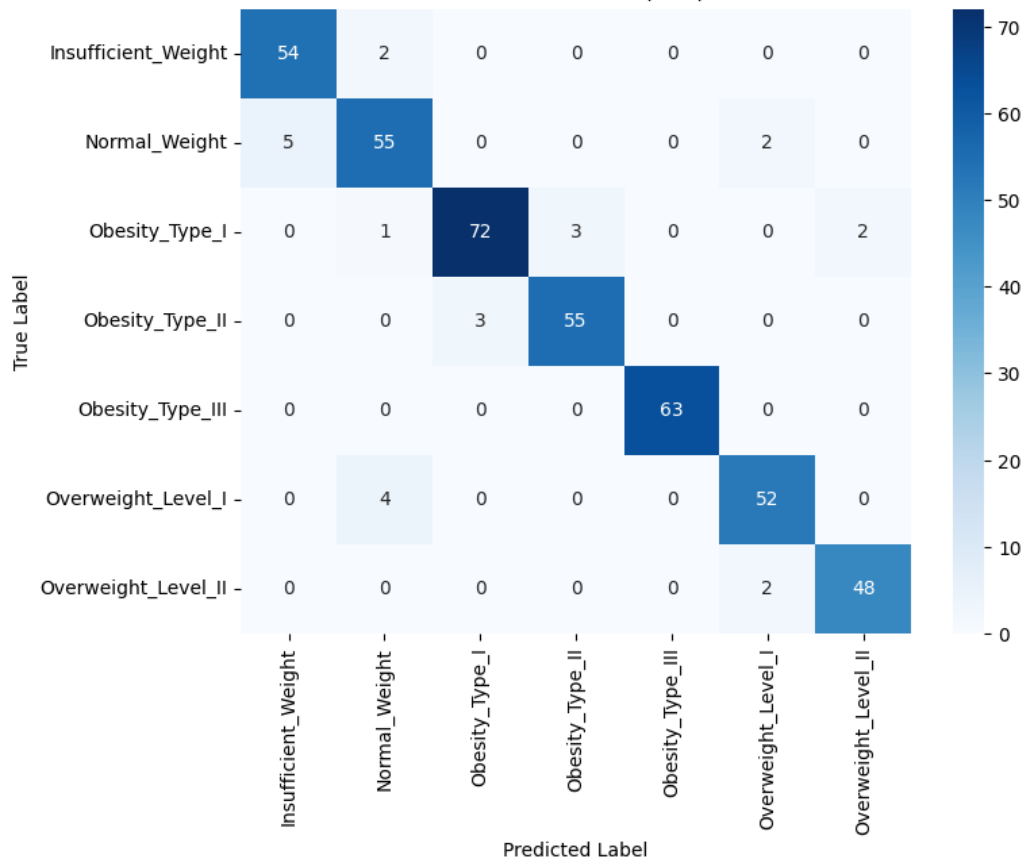
Output:

```
Confusion matrix shape: (7, 7)
```

Plot confusion matrix (Gini)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=target_names,
            yticklabels=target_names)
plt.title('Confusion Matrix (Gini)')
plt.show()
```

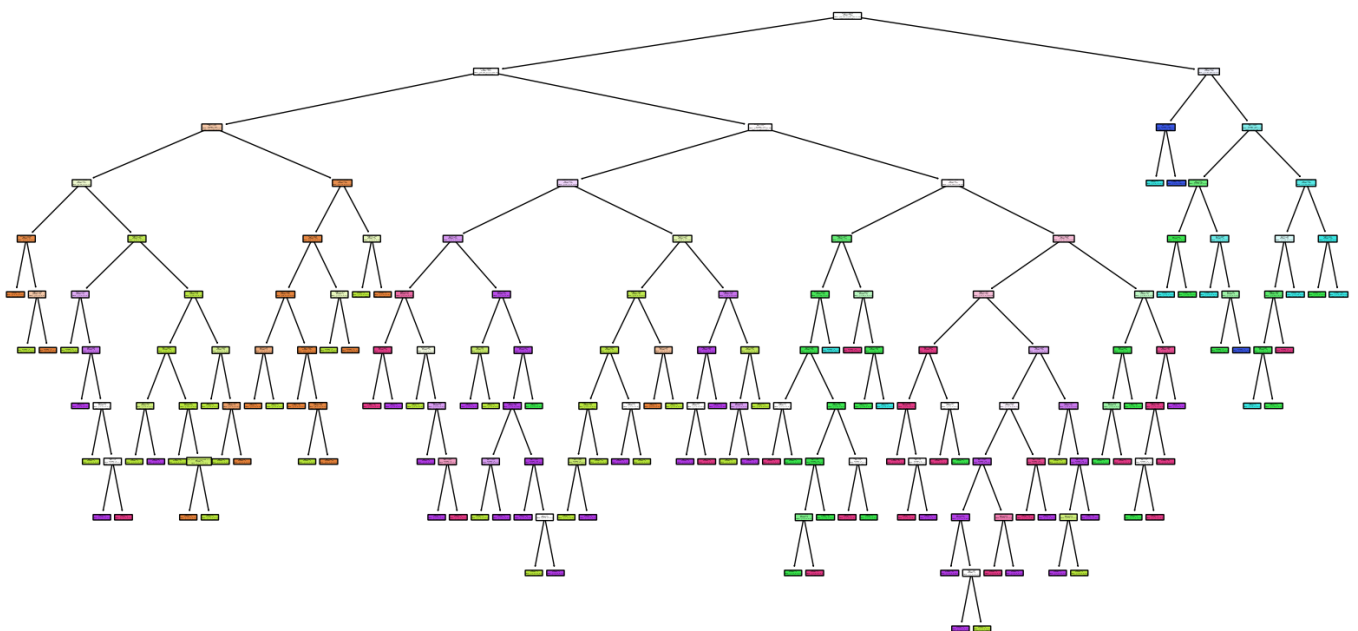
Confusion Matrix (Gini)



Visualize the Gini Decision Tree

```
print("Plotting the Decision Tree (Gini)...")
plt.figure(figsize=(20,10))
plot_tree(model, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.show()
```

Full Decision Tree (Gini)



Step 5: Training (Entropy)

Train and evaluate Decision Tree with Entropy

```
model_entropy = DecisionTreeClassifier(criterion='entropy')
model_entropy.fit(X_train, y_train)
y_pred_entropy = model_entropy.predict(X_test)
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
```

Output:

Accuracy (Entropy, Full Tree): 0.9598

Compute confusion matrix for Entropy

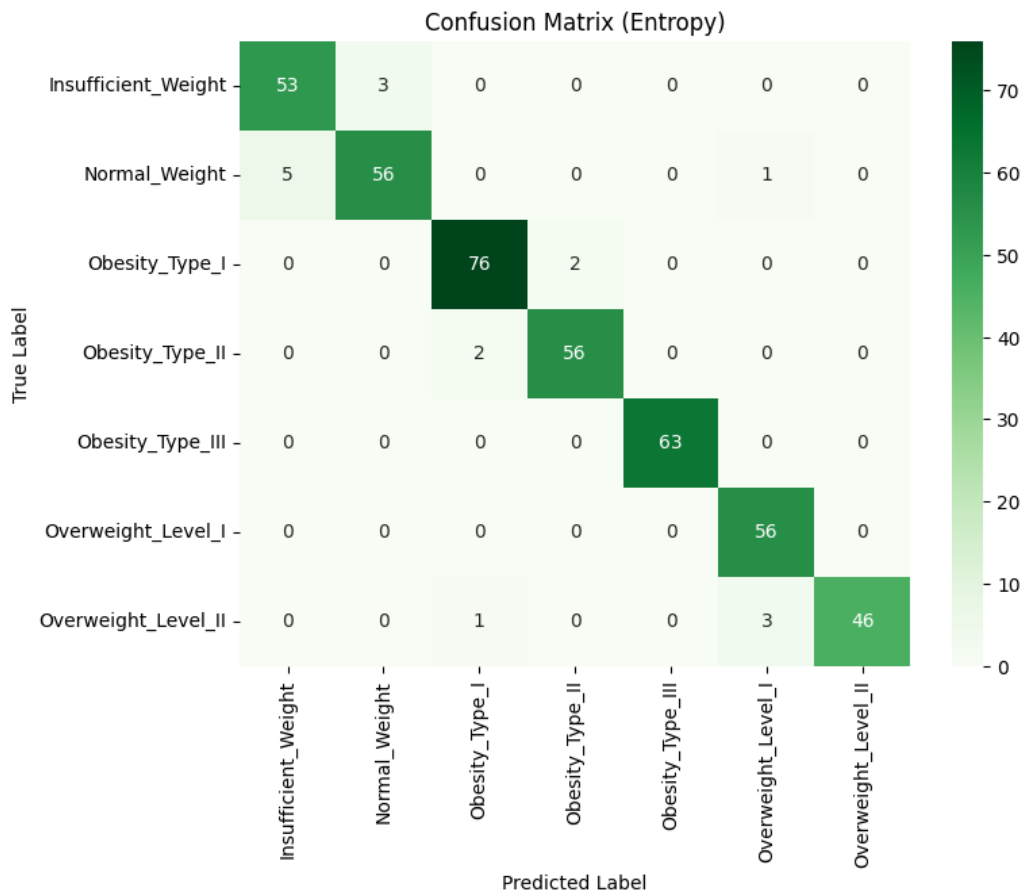
```
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
print("Confusion matrix shape:", cm_entropy.shape)
```

Output:

Confusion matrix shape: (7, 7)

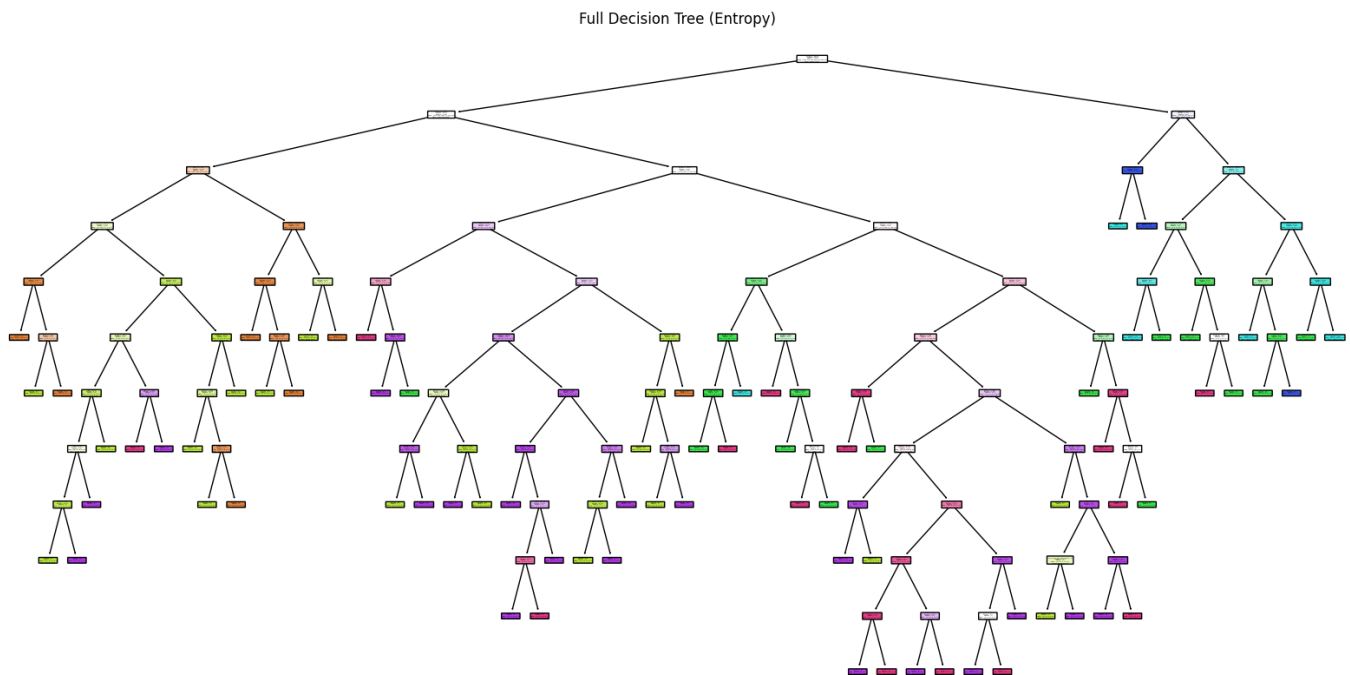
Plot confusion matrix (Entropy)

```
print("Displaying plot...")
plt.figure(figsize=(8,6))
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,
            yticklabels=target_names)
plt.show()
```



Visualize the Entropy Decision Tree

```
print("Plotting the Decision Tree (Entropy)...")
plot_tree(model_entropy, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.show()
```



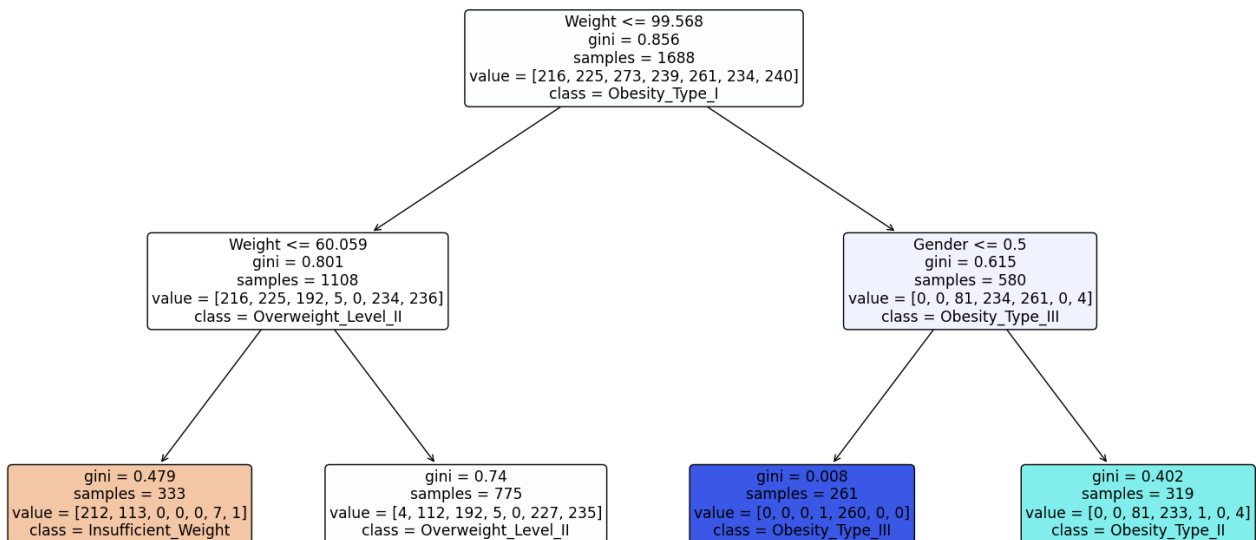
Step 6: Model Evaluation (Depth)

Analyze Decision Tree with Max Depth = 2

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

Max Depth: 2, Accuracy: 0.5432



Analyze Decision Tree with Max Depth = 4

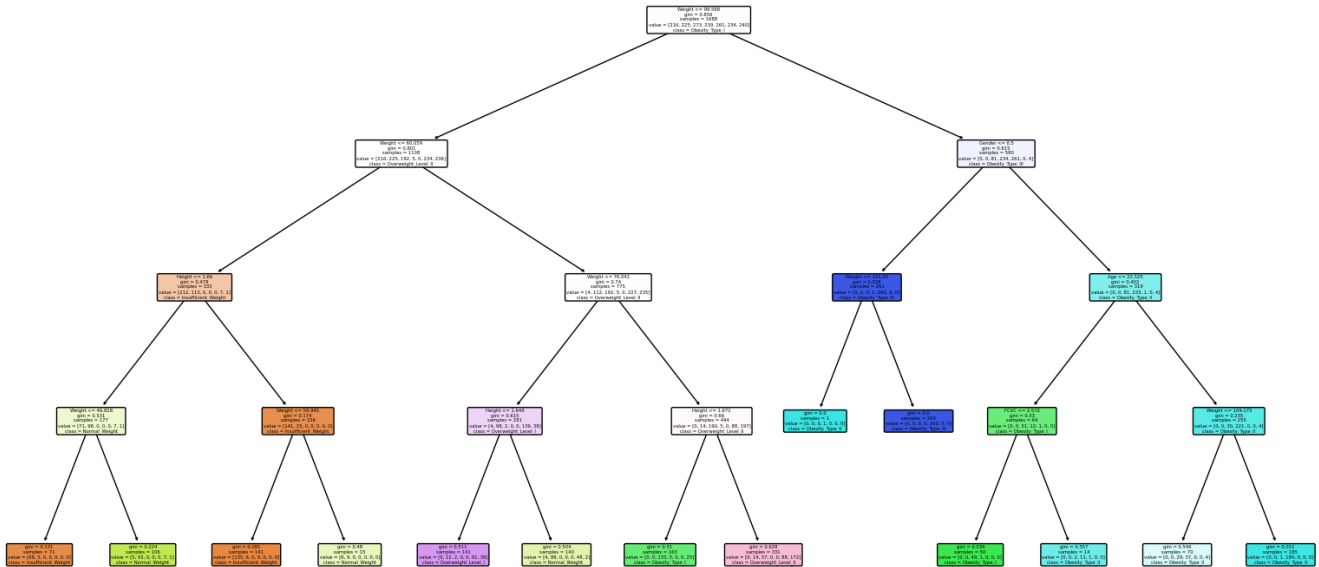
```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth: {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d})")
plt.show()
```

Output:

Max Depth: 4, Accuracy: 0.9433

Decision Trees - Practical Implementation

Decision Tree (Max Depth = 4)



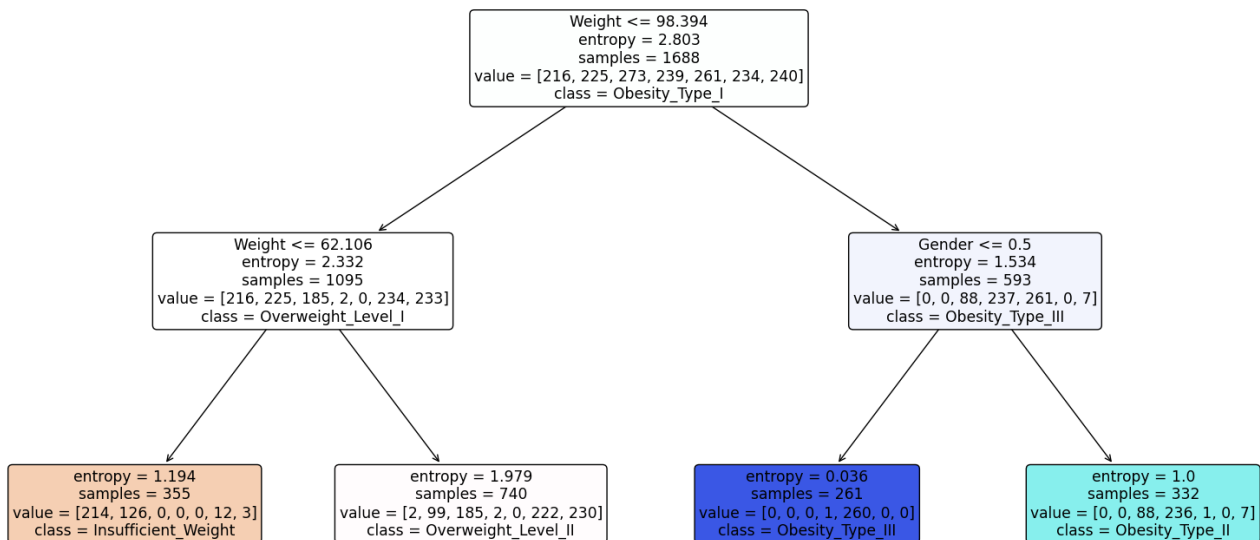
Analyze Decision Tree with Max Depth = 2 (Entropy)

```
d = 2
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

Max Depth (Entropy): 2, Accuracy: 0.9456

Decision Trees - Practical Implementation
Decision Tree (Max Depth = 2, Entropy)



Analyze Decision Tree with Max Depth = 4 (Entropy)

```
d = 4
clf = DecisionTreeClassifier(max_depth=d, criterion='entropy', random_state=42)
clf.fit(X_train, y_train)
acc = accuracy_score(y_test, clf.predict(X_test))
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
plt.figure(figsize=(20,10))
plot_tree(clf, feature_names=X.columns, class_names=target_names, filled=True,
          rounded=True)
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
plt.show()
```

Output:

Max Depth (Entropy): 4, Accuracy: 0.9598

Decision Trees - Practical Implementation

Decision Tree (Max Depth = 4, Entropy)

